

Joel Sivanish

Professor Kaur

Lab 6 – Write a Flask Chat App

2/26/2026

Summary:

The goal of this lab was to transition a low-level socket-based cryptographic system into a functional web application. By leveraging Flask-SocketIO, the system establishes a secure, persistent communication channel over WebSockets. The implementation demonstrates the practical application of Diffie-Hellman for key exchange, AES-CTR for encryption, HMAC-SHA256 for message integrity, and RSA for authentication and non-repudiation.

1. Architecture Comparison:

- **Communication Model Transition:** Lab 5 utilized raw TCP sockets, which required manual management of network streams and multi-client threading. Lab 6 adopts the HTTP/WebSocket model, allowing for a persistent, full-duplex connection that is more suitable for real-time web applications.
- **HTTP Overhead:** Unlike the raw packets in Lab 5, which consisted almost entirely of application payload, the HTTP/WebSocket model adds significant overhead. This includes HTTP headers (e.g., User-Agent, Upgrade) and WebSocket framing bits, which increase the packet size compared to the minimalist raw TCP segments.
- **Environment Setup:** The project was developed within a Python Virtual Environment (cybr_venv) to isolate dependencies like flask-socketio, eventlet, and pycryptodome, ensuring a consistent runtime environment regardless of system-level configurations.

2. Cryptographic Integration (RSA, AES, HMAC):

- **Integration Logic:** The functions from `rsa.py` and `cipher.py` were imported directly into the Flask server (`app.py`). The procedural command-line inputs from Lab 5 were replaced with WebSocket event handlers (`@socketio.on('message')`), which trigger the cryptographic modules whenever a user sends data.
- **Data Type Conversion:** Significant conversion was necessary. Python's cryptographic modules work with bytes, but WebSockets communicate via JSON, which only supports strings.

- **Base64 Implementation:** To bridge this, all encrypted blocks and RSA signatures were converted into Base64-encoded strings using `base64.b64encode()` before transmission, ensuring the binary data did not corrupt the JSON packet structure.

3. Diffie-Hellman Implementation:

- **Handshake Timing:** The Diffie-Hellman (DH) handshake is implemented as an automated event that occurs immediately upon the client's initial WebSocket 'connect' event.
- **Shared Secret Storage:** The derived Shared Secret is stored on the server side in a global dictionary (`client_sessions`), indexed by the client's unique Session ID (SID).
- **Security Justification:** This is secure because the Shared Secret itself is never transmitted; only the public DH components are exchanged over the network. By associating the secret with the sid, the server ensures that each user has a unique, isolated set of AES and HMAC keys, preventing session leakage between concurrent users.

4. The Logging Panel:

- **Visual Verification:** The interface features a Cryptographic Logging Panel that updates in real-time as messages are processed.
- **RSA & HMAC Identification:** Each message entry in the panel displays the Raw RSA Signature (a long Base64 string) and the HMAC Status (labeled "Pass" upon successful verification).
- **Critical Monitoring:** Visualizing these logs is essential for monitoring security and non-repudiation. The signature provides mathematical proof that a message originated from a specific sender (using their private key), while the HMAC status confirms that the message was not tampered with during transit.

5. Man-in-the-Middle (Web Context):

- **XSS Risks:** If the application is served over standard HTTP (rather than HTTPS), it is vulnerable to a Man-in-the-Middle attacker who can inject a malicious Cross-Site Scripting (XSS) script into the user's browser. This script could potentially access the client's memory or storage to steal an RSA private key if it were generated or stored locally by the browser.
- **Same-Origin Policy (SOP):** The Same-Origin Policy hinders security by being a "sandbox" only; while it prevents a site like `attacker.com` from directly reading your chat data, it offers no protection if the attacker successfully injects a script into the `chat.html` page. If the script is running from the "same origin" as the chat app, it has full access to all cryptographic secrets in the browser's memory.

Conclusion:

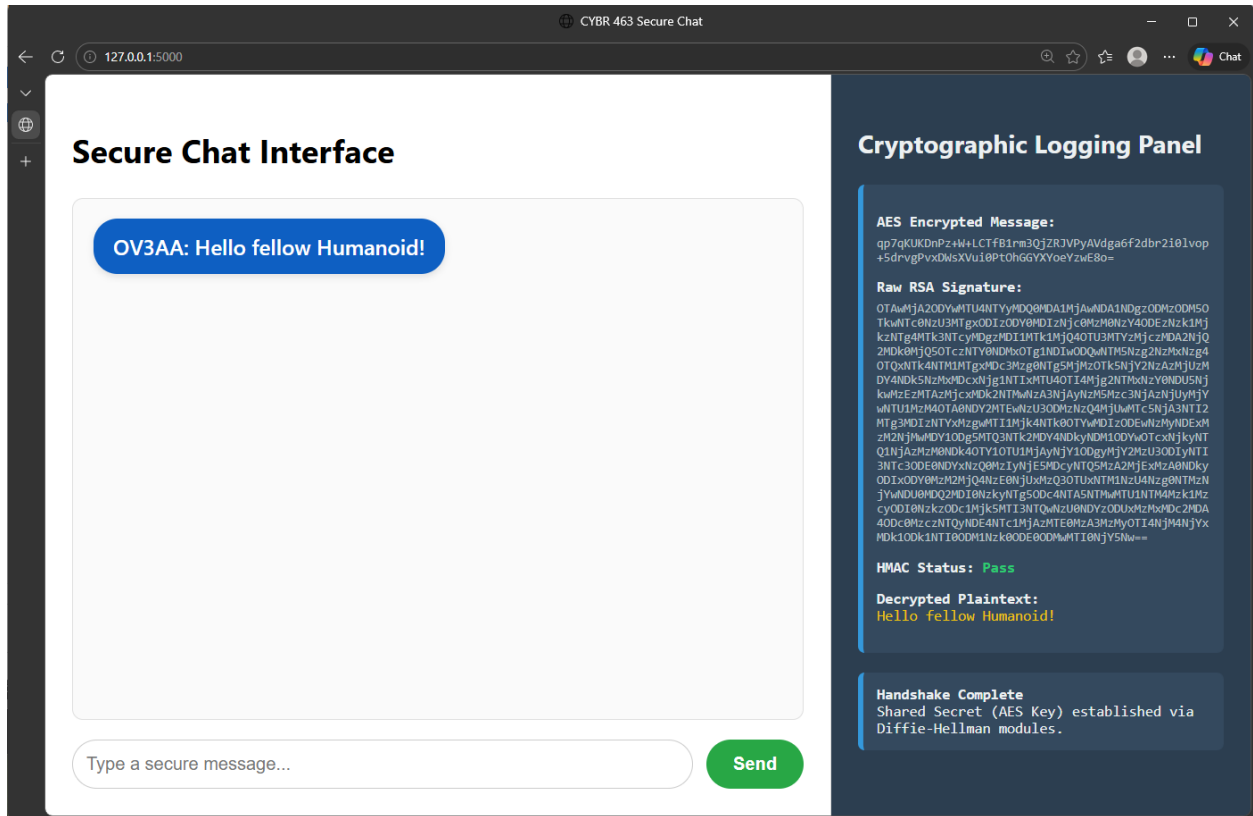
Lab 6 successfully demonstrates the application of applied cryptography within a modern web framework. By porting `diffiehellman.py`, `rsa.py`, and `cipher.py` to a Flask environment, the system provides a robust solution for Confidentiality, Integrity, and Non-repudiation in a multi-user chat environment.

Screenshots:

Server is running:

```
(cybr_venv) joel@DESKTOP-REQ0QHQ:/mnt/c/Users/Joel/Documents/School/EWU/Winter2026/CYBR463-AC/Lab/Lab6/Lab6$ python3 app
.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 570-678-301
127.0.0.1 - - [26/Feb/2026 14:15:29] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 14:15:29] "GET /socket.io/?EIO=4&transport=polling&t=PoS1NF7 HTTP/1.1" 200 -
Client Connected: xghNyP-tR7T0-COYAAAB
127.0.0.1 - - [26/Feb/2026 14:15:29] "POST /socket.io/?EIO=4&transport=polling&t=PoS1NF0&sid=MAiZBV6ew0nGwoyyAAAA HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 14:15:29] "GET /socket.io/?EIO=4&transport=polling&t=PoS1NF0.0&sid=MAiZBV6ew0nGwoyyAAAA HTTP/1.1" 200 -
127.0.0.1 - - [26/Feb/2026 14:15:29] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [26/Feb/2026 14:15:29] "POST /socket.io/?EIO=4&transport=polling&t=PoS1NFV&sid=MAiZBV6ew0nGwoyyAAAA HTTP/1.1" 200 -
REAL shared secret established for session: xghNyP-tR7T0-COYAAAB
127.0.0.1 - - [26/Feb/2026 14:15:29] "GET /socket.io/?EIO=4&transport=polling&t=PoS1NFU&sid=MAiZBV6ew0nGwoyyAAAA HTTP/1.1" 200 -
```

Chat UI, Single Session:



Logging Panel:

Chat UI, Multiple Sessions:

